```c
#include <stdio.h>
#include <stdlib.h>
#include <assert.h>
#include "bst.h"

bst_t *create_bst(void)
{
    return ((bst_t*)xcalloc(1, sizeof(bst_t)));
}

ret_t insert_bst(bst_t *p_bst, data_t new_data)
{
    bst_node_t *p_new_node = NULL;
    bst_node_t *p_run = NULL;

    p_new_node = get_bst_node(new_data);
    p_run = p_bst->p_root_node;

    if(!p_run)
    {
        p_bst->p_root_node = p_new_node;
        p_bst->nr_elements++;
        return (SUCCESS);
    }

    while(TRUE)
    {
        if(new_data <= p_run->data)
        {
            if(p_run->left == NULL)
            {
                p_run->left = p_new_node;
                p_new_node->parent = p_run;
                p_bst->nr_elements++;
                return (SUCCESS);
            }
            else
            {
                p_run = p_run->left;
                continue;
            }
        }
        else
        {
```

```
        else
        {
            if(p_run->right == NULL)
            {
                p_run->right = p_new_node;
                p_new_node->parent = p_run;
                p_bst->nr_elements++;
                return (SUCCESS);
            }
            else
            {
                p_run = p_run->right;
                continue;
            }
        }
    }
}

bool search_bst(bst_t *p_bst, data_t search_data)
{
    return (search_bst_node(p_bst->p_root_node, search_data) != NULL);
}

void inorder_r(bst_t *p_bst)
{
    __inorder_r(p_bst->p_root_node);
}

void preorder_r(bst_t *p_bst)
{
    __preorder_r(p_bst->p_root_node);
}

void postorder_r(bst_t *p_bst)
{
    __postorder_r(p_bst->p_root_node);
}
```

```c

void inorder_nrc(bst_t *p_bst)
{
    bst_node_t *p_run = NULL;
    stack_t *p_stack = NULL;

    p_stack = create_bstptr_stack();
    p_run = p_bst->p_root_node;

    printf("[start]\n");

    while(TRUE)
    {
        while(p_run != NULL)
        {
            push_bstptr(p_stack, p_run);
            p_run = p_run->left;
        }

        if(pop_bstptr(p_stack, &p_run) == STACK_EMPTY)
            break;

        printf("[%d]<->", p_run->data);
        p_run = p_run->right;
    }

    printf("[end]\n");
    destroy_bstptr_stack(&p_stack);
}

```

```c

void preorder_nrc(bst_t *p_bst)
{
    bst_node_t *p_run = NULL;
    stack_t *p_stack = NULL;

    p_stack = create_bstptr_stack();
    p_run = p_bst->p_root_node;

    while(TRUE)
    {
        while(p_run != NULL)
        {
            printf("[%d]<->", p_run->data);
            push_bstptr(p_stack, p_run);
            p_run = p_run->left;
        }

        if(pop_bstptr(p_stack, &p_run) == STACK_EMPTY)
            break;

        p_run = p_run->right;
    }

    printf("[end]\n");
    destroy_bstptr_stack(&p_stack);
}

```

```c
void postorder_nrc(bst_t *p_bst)
{
    bst_node_t *p_run = NULL;
    stack_t *p_stack = NULL;
    color_t color;

    p_stack = create_bstptr_stack();
    p_run = p_bst->p_root_node;

    printf("[beg]<->");

    while(TRUE)
    {
        while(p_run != NULL)
        {
            push_bstptr(p_stack, p_run);
            set_bstptr_top_color(p_stack, GREY);
            p_run = p_run->left;
        }

        if(get_bstptr_top_color(p_stack, &color) == STACK_EMPTY)
            break;

        if(color == GREY)
        {
            set_bstptr_top_color(p_stack, BLACK);
            top_bstptr(p_stack, &p_run);
            p_run = p_run->right;
            continue;
        }
        else if(color == BLACK)
        {
            pop_bstptr(p_stack, &p_run);
            printf("[%d]<->", p_run->data);
            p_run = NULL;
        }
    }

    puts("[end]");
    destroy_bstptr_stack(&p_stack);
}
```

```c
ret_t maximum_bst(bst_t *p_bst, data_t *p_max_data)
{
    bst_node_t *p_run = NULL;

    p_run = p_bst->p_root_node;
    if(p_run == NULL)
        return (BST_EMPTY);

    while(p_run->right != NULL)
        p_run = p_run->right;

    *p_max_data = p_run->data;

    return (SUCCESS);
}

ret_t minimum_bst(bst_t *p_bst, data_t *p_min_data)
{
    bst_node_t *p_run = NULL;

    p_run = p_bst->p_root_node;
    if(p_run == NULL)
        return (BST_EMPTY);

    while(p_run->left != NULL)
        p_run = p_run->left;

    *p_min_data = p_run->data;

    return (SUCCESS);
}
```

```c
ret_t inorder_successor(bst_t *p_bst, data_t e_data, data_t *p_succ_data)
{
    bst_node_t *pe_node = NULL;
    bst_node_t *p_succ_node = NULL;

    pe_node = search_bst_node(p_bst->p_root_node, e_data);
    if(pe_node == NULL)
        return (BST_DATA_NOT_FOUND);

    p_succ_node = inorder_successor_node(pe_node);
    if(p_succ_node != NULL)
    {
        *p_succ_data = p_succ_node->data;
        return (SUCCESS);
    }

    return (BST_NO_SUCCESSOR);
}

ret_t inroder_predecessor(bst_t *p_bst, data_t e_data, data_t *p_pred_data)
{
    bst_node_t *pe_node = NULL;
    bst_node_t *p_pred_node = NULL;

    pe_node = search_bst_node(p_bst->p_root_node, e_data);
    if(pe_node == NULL)
        return (BST_DATA_NOT_FOUND);

    p_pred_node = inorder_predecessor_node(pe_node);
    if(p_pred_node != NULL)
    {
        *p_pred_data = p_pred_node->data;
        return (SUCCESS);
    }

    return (BST_NO_PREDECESSOR);
}
```

```c
ret_t remove_bst(bst_t *p_bst, data_t d_data)
{
    bst_node_t *z = NULL;   /* Address of node to be deleted */
    bst_node_t *y = NULL;   /* Address of successor node */

    z = search_bst_node(p_bst->p_root_node, d_data);
    if(z == NULL)
        return (BST_DATA_NOT_FOUND);

    if(z->left == NULL)
        transplant_bst(p_bst, z, z->right);
    else if(z->right == NULL)
        transplant_bst(p_bst, z, z->left);
    else
    {
        y = inorder_successor_node(z);

        if(z->right != y)
        {
            transplant_bst(p_bst, y, y->right);
            y->right = z->right;
            y->right->parent = y;
        }

        transplant_bst(p_bst, z, y);
        y->left = z->left;
        y->left->parent = y;
    }

    free(z);
    z = NULL;

    p_bst->nr_elements--;

    return (SUCCESS);
}
```

```c
ret_t remove_bst_extended(bst_t *p_bst, data_t r_data)
{
    bst_node_t *p_bst_delete_node = NULL;
    bst_node_t *z = NULL;
    bst_node_t *y = NULL;

    p_bst_delete_node = search_bst_node(p_bst->p_root_node, r_data);
    if(p_bst_delete_node == NULL)
        return (BST_DATA_NOT_FOUND);

    z = p_bst_delete_node;

    if(z->left == NULL)
    {
        if(z->parent == NULL)
            p_bst->p_root_node = z->right;
        else if(z->parent->left == z)
            z->parent->left = z->right;
        else
            z->parent->right = z->right;

        if(z->right != NULL)
            z->right->parent = z->parent;
    }
    else if(z->right == NULL)
    {
        if(z->parent == NULL)
            p_bst->p_root_node = z->left;
        else if(z->parent->left == z)
            z->parent->left = z->left;
        else
            z->parent->right = z->left;

        if(z->left != NULL)
            z->left->parent = z->parent;
    }
    else
    {
```

```
    else
    {
        y = z->right;
        while(y->left != NULL)
            y = y->left;

        if(z->right != y)
        {
            if(y == y->parent->left)
                y->parent->left = y->right;
            else
                y->parent->right = y->right;

            if(y->right != NULL)
                y->right->parent = y->parent;

            y->right = z->right;
            y->right->parent = y;
        }

        if(z->parent == NULL)
            p_bst->p_root_node = y;
        else if(z == z->parent->left)
            z->parent->left = y;
        else
            z->parent->right = y;

        y->parent = z->parent;
        y->left = z->left;
        y->left->parent = y;
    }

    free(z);
    z = NULL;

    p_bst->nr_elements--;

    return (SUCCESS);
}
```

```c
ret_t destroy_bst(bst_t **pp_bst)
{
    bst_t *p_bst = NULL;

    p_bst = *pp_bst;
    __destroy_bst_r(p_bst->p_root_node);
    free(p_bst);
    *pp_bst = NULL;

    return (SUCCESS);
}

/* Auxillary */
bst_node_t *search_bst_node(bst_node_t *p_root_node, data_t search_data)
{
    bst_node_t *p_run = NULL;

    p_run = p_root_node;

    while(p_run != NULL)
    {
        if(p_run->data == search_data)
            break;
        else if(p_run->data < search_data)
            p_run = p_run->right;
        else
            p_run = p_run->left;
    }

    return (p_run);
}

bst_node_t *get_bst_node(data_t new_data)
{
    bst_node_t *p_new_node = NULL;

    p_new_node = (bst_node_t*)xcalloc(1, sizeof(bst_node_t));
    p_new_node->data = new_data;

    return (p_new_node);
}
```

```c
bst_node_t *inorder_successor_node(bst_node_t *p_bst_node)
{
    bst_node_t *x = NULL;
    bst_node_t *y = NULL;
    bst_node_t *p_run = NULL;

    /* case 1 */
    x = p_bst_node;
    if(x->right != NULL)
    {
        p_run = x->right;
        while(p_run->left != NULL)
            p_run = p_run->left;

        return (p_run);
    }

    /* case 2 */
    y = x->parent;
    while(y != NULL && x == y->right)
    {
        x = y;
        y = x->parent;
    }

    return (y);
}
```

```c
bst_node_t *inorder_predecessor_node(bst_node_t *p_bst_node)
{
    bst_node_t *x = NULL;
    bst_node_t *y = NULL;
    bst_node_t *p_run = NULL;

    x = p_bst_node;

    /* case 1 */
    if(x->left != NULL)
    {
        p_run = x->left;
        while(p_run->right != NULL)
            p_run = p_run->right;

        return (p_run);
    }

    /* case 2 */
    y = x->parent;
    while(y != NULL && x == y->left)
    {
        x = y;
        y = x->parent;
    }

    return (y);
}

void __inorder_r(bst_node_t *p_root_node)
{
    if(p_root_node != NULL)
    {
        __inorder_r(p_root_node->left);
        printf("%d\n", p_root_node->data);
        __inorder_r(p_root_node->right);
    }
}
```

```c

void __preorder_r(bst_node_t *p_root_node)
{
    if(p_root_node != NULL)
    {
        printf("%d\n", p_root_node->data);
        __preorder_r(p_root_node->left);
        __preorder_r(p_root_node->right);
    }
}

void __postorder_r(bst_node_t *p_root_node)
{
    if(p_root_node != NULL)
    {
        __postorder_r(p_root_node->left);
        __postorder_r(p_root_node->right);
        printf("%d\n", p_root_node->data);
    }
}

void __destroy_bst_r(bst_node_t *p_root_node)
{
    if(p_root_node != NULL)
    {
        __destroy_bst_r(p_root_node->left);
        __destroy_bst_r(p_root_node->right);
        free(p_root_node);
    }
}

void transplant_bst(bst_t *p_bst, bst_node_t *u, bst_node_t *v)
{
    if(u->parent == NULL)
        p_bst->p_root_node = v;
    else if(u == u->parent->left)
        u->parent->left = v;
    else
        u->parent->right = v;

    if(v)
        v->parent = u->parent;
}
```

```c

/* BST auxillary - Stack interface */
stack_t *create_bstptr_stack()
{
    return create_bstptr_list();
}

ret_t push_bstptr(stack_t *p_stack, bst_node_t *p_bst_node)
{
    return insert_bstptr_end(p_stack, p_bst_node);
}

ret_t top_bstptr(stack_t *p_stack, bst_node_t **pp_bst_node)
{
    return examine_bstptr_end(p_stack, pp_bst_node);
}

ret_t pop_bstptr(stack_t *p_stack, bst_node_t **pp_bst_node)
{
    return examine_and_delete_bstptr_end(p_stack, pp_bst_node);
}

bool is_bstptr_stack_empty(stack_t *p_stack)
{
    return is_bstptr_list_empty(p_stack);
}

ret_t get_bstptr_top_color(stack_t *p_stack, color_t *p_top_color)
{
    if(is_bstptr_stack_empty(p_stack))
        return (STACK_EMPTY);
    *p_top_color = p_stack->prev->color;
    return (SUCCESS);
}

ret_t set_bstptr_top_color(stack_t *p_stack, color_t new_color)
{
    if(is_bstptr_stack_empty(p_stack))
        return (STACK_EMPTY);
    p_stack->prev->color = new_color;
    return (SUCCESS);
}
```

```c
ret_t destroy_bstptr_stack(stack_t **pp_stack)
{
    return destroy_bstptr_list(pp_stack);
}

/* BST auxillary - List interface */
list_t *create_bstptr_list()
{
    list_t *p_new_list = NULL;

    p_new_list = get_bstptr_node(NULL);
    p_new_list->next = p_new_list;
    p_new_list->prev = p_new_list;

    return (p_new_list);
}

ret_t insert_bstptr_end(list_t *p_list, bst_node_t *p_bst_node)
{
    bstptr_node_t *p_new_bstptr_node = NULL;

    p_new_bstptr_node = get_bstptr_node(p_bst_node);
    generic_bstptr_insert(p_list->prev, p_new_bstptr_node, p_list);

    return (SUCCESS);
}

ret_t examine_bstptr_end(list_t *p_list, bst_node_t **pp_bst_node)
{
    if(is_bstptr_list_empty(p_list))
        return (LIST_EMPTY);

    *pp_bst_node = p_list->prev->p_bst_node;
    return (SUCCESS);

}
```

```c

ret_t examine_and_delete_bstptr_end(list_t *p_list, bst_node_t **pp_bst_node)
{
    if(is_bstptr_list_empty(p_list))
        return (LIST_EMPTY);

    *pp_bst_node = p_list->prev->p_bst_node;
    generic_bstptr_delete(p_list->prev);

    return (SUCCESS);
}

bool is_bstptr_list_empty(list_t *p_list)
{
    return (p_list->next == p_list && p_list->prev == p_list);
}

ret_t destroy_bstptr_list(list_t **pp_list)
{
    bstptr_node_t *p_bstptr_run = NULL;
    bstptr_node_t *p_bstptr_run_next = NULL;
    list_t *p_list = NULL;

    p_list = *pp_list;
    p_bstptr_run = p_list->next;
    while(p_bstptr_run != p_list)
    {
        p_bstptr_run_next = p_bstptr_run->next;
        free(p_bstptr_run);
        p_bstptr_run = p_bstptr_run_next;
    }

    free(p_list);
    *pp_list = NULL;

    return (SUCCESS);

}

```

```c
/* Bst auxillary - List interface - List auxillary */
void generic_bstptr_insert(bstptr_node_t *p_beg, bstptr_node_t *p_mid, bstptr_node_t *p_end)
{
    p_mid->next = p_end;
    p_mid->prev = p_beg;
    p_beg->next = p_mid;
    p_end->prev = p_mid;
}

void generic_bstptr_delete(bstptr_node_t *p_bstptr_node)
{
    p_bstptr_node->prev->next = p_bstptr_node->next;
    p_bstptr_node->next->prev = p_bstptr_node->prev;
    free(p_bstptr_node);
    p_bstptr_node = NULL;
}

bstptr_node_t *search_bstptr_node(list_t *p_list, bst_node_t *p_bst_node)
{
    bstptr_node_t *p_bstptr_run = NULL;

    p_bstptr_run = p_list->next;
    while(p_bstptr_run != p_list)
    {
        if(p_bstptr_run->p_bst_node == p_bst_node)
            return (p_bstptr_run);
        p_bstptr_run = p_bstptr_run->next;
    }

    return (NULL);
}

bstptr_node_t *get_bstptr_node(bst_node_t *p_bst_node)
{
    bstptr_node_t *p_bstptr_new_node = NULL;

    p_bstptr_new_node = (bstptr_node_t*)xcalloc(1, sizeof(bstptr_node_t));
    p_bstptr_new_node->p_bst_node = p_bst_node;
    p_bstptr_new_node->color = WHITE;

    return (p_bstptr_new_node);
}
```

```
674
675 void *xcalloc(size_t n, size_t s)
676 {
677     void *p = NULL;
678
679     p = calloc(n, s);
680     assert(p);
681
682     return (p);
683 }
684
```